

```
self.brand = brand
self.location = location
--- snipped remaining initialization of TV ---
...
```

In all methods, parameters are local variables, so they literally go away when the method ends. For example, in the `__init__()` method of the `TV` class shown here, `brand` and `location` are local variables that will disappear when the method ends. However, we often want to save values that are passed in via parameters to use them in other methods.

In order to allow an object to remember initial values, the standard approach is to store any values passed in into instance variables. Since instance variables have object scope, they can be used in other methods in the class. The Python convention is that the name of the instance variable should be the same as the parameter name, but prefixed with `self` and a period:

```
def __init__(self, someVariableName):
    self.someVariableName = someVariableName
```

In the `TV` class, the line after the `def` statement tells Python to take the value of the `brand` parameter and assign it to an instance variable named `self.brand`. The next line does the same thing with the `location` parameter and the instance variable `self.location`. After these assignments, we can use `self.brand` and `self.location` in other methods.

Using this approach, we can create multiple objects from the same class but start each off with different data. So, we can create our two `TV` objects like this:

```
oTV1 = TV('Samsung', 'Family room')
oTV2 = TV('Sony', 'Bedroom')
```

When executing the first line, Python first allocates space for a `TV` object. Then it rearranges the arguments as discussed in the previous section and calls the `__init__()` method of the `TV` class with three arguments: the newly allocated `oTV1` object, the `brand`, and the `location`.